

AI Project Report

The environment and agent I decided to develop for my final project is based around the historical viking boardgame, “Hnefetafl”. The Viking Age itself is fairly well-documented, however there are a lot of limitations in both the quantity and nature of these records. As such, Hnefetafl itself is a game with a ruleset that’s not very well documented, and we don’t fully know exactly how it was played. However, there are many variations of the Hnefetafl family of games known as “Tafl games” or “table games” in English. One such variation of Hnefetafl is known as Tablut, and it originates from the Sami people of Scandanavia.

Tablut is a much more well documented version of Hnefetafl with a very clear rule set. Each of the Tafl games is asymmetric, meaning the two sides have a different number of pieces, separate objectives, and varying strategies for winning. The games all vary in their board size, number of pieces, rules, and layout of the pieces. Tablut is played on a 9x9 grid, and is played by two players.

One player is the defending side. They have 9 pieces in total; 1 king and 8 defenders. The king is positioned in the center square of the board, called the castle or the throne. Surrounding the king in a cross-like pattern are its 8 defenders. These pieces are used to strategically defend the king in achieving their objective, capturing attackers, and often sacrificing themselves to save the king. On the attackers’ side, they simply have 16 pieces that are positioned symmetrically somewhere around the edges of the board.

Each side has a very different objective to win the game. On the defenders side, the player must get their king to any of the spaces on the outside edges of the board. The goal of the attackers is simply to capture the king before he escapes. All pieces, including the king, move like a rook in chess — They can move any number of spaces in a straight line, pieces cannot jump over others, and a piece must stop its path if blocked by another piece.

A piece is captured when it is surrounded on two opposite sides by enemy pieces. However, there are a few special exceptions to this rule. For one, the central square (the castle/throne) is considered hostile to all pieces except the king, meaning a piece can be captured if it is adjacent to the castle and surrounded on the other side by an enemy piece. However, if the king is currently inside of the castle, it cannot be captured unless it is surrounded on all 4 sides by attackers. If the king moves out of the castle and is adjacent to it, he can be captured if surrounded on three sides (the castle acting as the fourth). If the king is not

adjacent to or within the castle, he is susceptible to capture just like any other piece on the board.

This leads into the PEAS assessment for my project. As the game is played by two players, it was most appropriate to develop it in a PettingZoo environment, so the PEAS assessment is mostly standard for a typical PettingZoo board game environment involving two players.

The performance measure is simply based on whether the defender wins or loses, with a win being worth 1 point and a loss being worth -1 point. The environment itself is fully observable, meaning the entire board and the positions of all pieces are visible to the agent. The uncertainty is deterministic, so each action has a predictable outcome based on the rules of the game. It's sequential, the agent must make a series of moves and the sequence of actions impacts the outcome. The stability is static, because the environment doesn't change unless the agent or opponent makes a move. It's discrete, it operates on a grid-based board with discrete turns and movement. The game is played by two participants, so it's a multi-agent environment. And lastly, the knowledge of the rules of Hnefatafl are fully known to the agent and there are no unknown variables.

Lastly, we have the actions and percepts. The only action available to the agent is the ability to move. As discussed earlier, an agent can move any piece it chooses any number of spaces horizontally or vertically, unless its path is blocked by another piece similarly to a rook in Chess. As for the percepts, there is the board state which is the current positions of all of the pieces, as well as what spaces are unoccupied. The environment gives a list of all the legal moves based on the game's rules, and we also have the capture status in which an agent can perceive when a piece has been captured and is no longer on the board.

To start the process of developing my search algorithm for my environment, I first created a random agent. The random agent simply picks a legal move from the list of available legal moves to make at random. Using the previous assignment of the connect four agent as an example, I decided to go with the minimax algorithm with alpha-beta pruning to allow the agent to explore its possible moves, simulate the outcomes, and evaluate these outcomes based on a scoring evaluation. This allows the agent to prioritize moves that increase its likelihood of winning the match, and minimizing the moves that put it in a state of disadvantage against its opponent.

This is around the turning point where things began getting a little more interesting and curious. Because the Tafl games are all asymmetric, there are two very different goals for the agent to be potentially trying to achieve depending on which side it's on. I could either choose to

implement minimax for the defender and keep the attacker random, or I could implement minimax for the attacker and make the defender random. Both would use the same algorithm, but with very different evaluation functions.

I decided to go with implementing minimax with alpha beta pruning in a defender agent against a random attacker agent at first. This is because, I knew that the defender agent's evaluation function would be much more simple and straight forward. All it needed to do was make moves that give it the ability to move the king closer to the edges of the board. As such, the evaluation function gave a better score depending on two major factors — If a future move makes it win the game, or if a move simply reduces the king's manhattan distance to the edges of the board.

The alpha-beta pruning optimization came into play because there are many possible moves in Tablut. Each piece can move any number of spaces horizontally or vertically, and there are 81 spaces in total with 25 of the spaces being occupied at the start of the game by both players' pieces. It was extremely important for the search algorithm to be efficient, and alpha-beta pruning is able to reduce unnecessary evaluations and makes it more efficient without sacrificing too much accuracy.

Having implemented minimax with alpha-beta pruning into the defender agent, it was extremely effective when going up against the random attacker agent. However, it was in my opinion, a little bit *too* effective. Having run hundreds of games with the random attacker agent going up against my search algorithm defender, the attacker wasn't able to win a single match. Statistically, the odds of an attacker randomly moving two of its pieces in such a way that it can capture the king was extremely low, especially since the king was constantly moving to try to get closer to the edges. I knew that in order to get proper statistical results from my agent's performance, I needed a more intelligent attacker agent as well.

With this goal in mind, I once again implemented the minimax algorithm with alpha-beta pruning into the attacker agent. However, the evaluation function for the attacker was much more complicated. I had a plan in mind for getting better results, which I'll get into later, but I wanted to make sure the attacker agent covers all of its bases. The evaluation function for the attacker took many things into consideration, such as rewarding the agent for decreasing the number of defenders on the board. It also tried to keep the king as far away from the edges of the board as possible, getting rewarded for keeping the king near the center. The attacker's proximity to the king was also taken into consideration, with higher value moves putting the attacker closer to the king. It also took note of how many defenders it could potentially capture from a specific position, so even if it's not able to do so on its current turn, it

could put itself in a position of great advantage to attack multiple opponents. Lastly, it was rewarded for blocking the king's escape routes at different positions.

Having implemented this, the attacker agent was now dominating the defender just as I had planned. My goal was to create a strong opponent in the attacker that I could then use as the new baseline for testing my defender agent against, as opposed to going up against a fully random opponent. However, in having implemented the minimax algorithm in both opponents, I created somewhat of an issue.

Because of their reliance on minimax with alpha-beta pruning, both agents' moves were completely deterministic. I noticed this when I was manually reviewing their matches and saw that they were making the exact same moves every time, and I was ending up with the same results. I had to come up with a way to introduce some degree of randomness into the agents while keeping their intelligence.

I decided that I was only going to implement randomness into one of the two agents as opposed to both. Because of this, I decided that it might be more beneficial to implement the randomness into the attacker agent. It was already much more intelligent than the defender, as the defender was extremely simple and I could easily build it up. Rather than go with a completely new algorithm, I thought of my own solution to the problem.

The method I chose to add randomness was to create an array of the "best moves" in my attacker agent. Anytime a move had a higher evaluation score than the current highest score, it would append that move to the array of best moves. The `selection_action` method of my agent's class would then select one of these moves at random. However, I didn't want a 'bad' move to have the same likeliness to be selected as a good move. To account for this, I weighted each of the best moves depending on how high their score was.

This weight was calculated using softmax – a way of representing the likelihood of choosing a move based on its evaluation score that's often used in neural networks. In my agent, the softmax output is used directly to sample a move probabilistically. The moves' scores and probabilities of being selected are exponentially related, meaning a very low scoring move is much less likely to be selected than a really good move. To me, this was similar to how a very high level player can make large mistakes, however rare it may be.

With this implementation in my attacker agent, we're avoiding deterministic behavior while still biasing towards good moves. I also implemented temperature scaling, much like with the policy search algorithm, to adjust the degree to which the attacker is more exploratory and random, but I kept the temperature pretty low for the most part to avoid too much randomness.

I still wanted the attacker to be intelligent and make good moves, because I wanted a tough opponent to put my defender up against.

The attacker was now much stronger and would beat the defender 90% of the time in all of the games I ran at this point. So as I planned to do from the start, I used this upgraded attacker as the new baseline and began developing new iterations of the defender agent. For the first new version of the defender agent, I made its evaluation function take a lot more into consideration.

Rather than having the defender only tunnel in on minimizing the king's distance to the edges of the board, I also made the defender utilize its other pieces a lot more. I did this by rewarding the agent for having more defenders near the king, and keeping the amount of defenders as high as possible. It is rewarded for getting the king into a position with high mobility, meaning there are many spaces it can move to with very little threat. It's also rewarded for how many attackers they remove from the board, or how many they have the potential to capture much like how the attackers put themselves into positions where they can have advantage against defenders. The defender is penalized, however, for how many immediate threats the king has against him.

With these strategies implemented into the evaluation function, the defender was winning quite a bit more than before, but it was still only winning around 30-40% of its games. I still wasn't satisfied with this result, so I developed a third and final defender agent.

For the final version of the defender agent, I decided to research some game winning strategies for defenders in Tablut. I came across a strategy that revolves around the idea of following a set of priorities anytime you're making a move. The first and highest priority is to never move the king, unless you're removed enough enemy pawns to absolutely ensure victory. Next is to take an enemy pawn anytime you're able to. The enemies outnumber the defenders by a lot, so it's important to stop them from ever getting advantageous positions. If you aren't able to capture any enemy pawns, it's best to keep your pawns back in the starting position. The starting position is incredibly powerful, and you can pretty much take any enemy pawn if it gets moved adjacent to your own. If you can't take an enemy pawn and you're already in the starting position, then you need to make the safest move possible. This is basically to stall for time and force the enemy into another position where you can capture them.

The results of this strategy were very effective, with defenders now winning 80-90% of their games with an average length of only 26.1 moves. Finally, I had an agent that was able to regularly win against a tough opponent and I finally started to feel satisfied with my results.

Using the new attacker as the baseline rather than a random opponent or something like that was quite a lot more work, but it paid off immensely.

Results from 25 Games Per Pairing:

Defender Agent	Attacker Agent	Defender Win %	Attacker Win %	Average Length
Random	Random	76%	24%	115 Moves
Version 1	Random	100%	0%	20 Moves
Version 1	Version 1	20%	80%	35 Moves
Version 2	Version 1	32%	68%	44 Moves
Version 3	Version 1	80%	20%	28 Moves

There are a lot of things to take away from the results. For one, when both sides were selecting actions at random, the defender was still winning the majority of the games. This is because like I mentioned earlier, it's statistically much more unlikely for the attacker to position two of its pieces perfectly to capture the king, whereas all the defender needs to do is move the king to one of the board's edges.

The asymmetry is even further highlighted when the defender agent, having implemented minimax with alpha-beta pruning, has a very basic but clear strategy for winning the game. It won 100% of the 25 games with ease, being able to do so in 20 moves on average. It's clear that I made the right decision in further developing the attacker agent in order to create a tougher opponent.

The new version of the attacker played much more aggressively, and it took many things into account in order to give itself the advantage with every move. It especially favors being in positions where it's able to attack the king, or at least put it in danger. After implementing this, the attacker was winning the majority of its games, and the number of moves increased as both sides were using more complex strategies.

With the defender only focusing on getting the king as close to the edges as possible, the attacker won the majority of the matches in around 35 moves. The upgraded version of the defender would prove to be a bit more effective against the attacker, but it still wasn't good

enough. It wasn't until I implemented the final version of the defender using a proper game winning strategy that it was able to reach its peak.

This project taught me a lot about training an AI agent against a smarter agent, which wasn't really something that we did in any of our assignments. I had to find a way to create an intelligent attacker that has some degree of randomness, so as to not make every match have the same moves and outcomes with both agents utilizing minimax with alpha-beta pruning. I learned the value of having a tough adversary and building up an agent to deal with their opponent. If I were to develop this project further, I imagine I would implement the randomness with the same weight system to the defender and remove it from the attacker, and then try to focus on the attacker and make its evaluation function better and try to get it back on top.